

NOTES

Database Functions



Database Functions

Functions

What are Functions?

Functions are predefined operations that perform calculations, manipulate data, and return results. They simplify complex queries and automate repetitive tasks, making data extraction and analysis more efficient. SQL functions can be categorised mainly into aggregate functions and scalar functions.

Why Use Functions?

Functions are essential for several reasons:

- **Efficiency:** They allow for quick calculations and data manipulation without the need for extensive coding.
- **Simplicity:** Functions can encapsulate complex logic into a single callable unit, making queries easier to read and maintain.
- **Data Analysis:** Aggregate functions enable users to summarise large datasets, providing insights through statistical measures.

How to Use Functions?

Functions can be utilised in various parts of a SQL query, including the SELECT, WHERE, and ORDER BY clauses. They are categorised into several types based on their functionality, such as aggregate functions, string functions, date functions, and mathematical functions.

Types of SQL Functions

Aggregate Functions

Aggregate functions perform calculations on a set of values and return a single value. They are typically used in conjunction with the GROUP BY clause to summarise data. Common aggregate functions include:

- **SUM():** Calculates the total sum of a numeric column.
- **AVG():** Computes the average value of a numeric column.
- **COUNT():** Returns the number of rows in a set, ignoring NULL values (except for COUNT(*) which counts all rows).
- **MIN():** Finds the smallest value in a selected column.
- **MAX():** Identifies the largest value in a selected column.

Table Creation and Data Insertion Queries:

Create Products Table:

Unset

```
CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(50),
  Category VARCHAR(50),
  Price DECIMAL(10, 2)
);
```

Insert Data into Products:

Unset

```
INSERT INTO Products (ProductID, ProductName, Category, Price)
VALUES
(101, 'Laptop', 'Electronics', 1000),
(102, 'Smartphone', 'Electronics', 700),
(103, 'Tablet', 'Electronics', 400),
```

```
(104, 'Smartwatch', 'Electronics', 200),  
(105, 'Headphones', 'Accessories', 50);
```

Create Sales Table:

Unset

```
CREATE TABLE Sales (  
    SaleID INT PRIMARY KEY,  
    ProductID INT,  
    Quantity INT,  
    SaleDate DATE,  
    SalePrice DECIMAL(10, 2),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

Insert Data into Sales:

Unset

```
INSERT INTO Sales (SaleID, ProductID, Quantity, SaleDate, SalePrice)  
VALUES  
(1, 101, 5, '2024-09-01', 150.00),  
(2, 102, 2, '2024-09-03', 300.00),  
(3, 103, 1, '2024-09-05', 500.00),  
(4, 104, 3, '2024-09-08', 200.00),  
(5, 101, 7, '2024-09-10', 120.00);
```

Aggregate Functions:

SUM() - Calculate the total revenue generated.

Unset

```
SELECT SUM(SalePrice * Quantity) AS TotalRevenue  
FROM Sales;
```

Output:

```

32
33 • SELECT SUM(SalePrice * Quantity) AS TotalRevenue
34 FROM Sales;

```

Result Grid	
	TotalRevenue
▶	3290.00

AVG() - Find the average price of products sold.

Unset

```

SELECT AVG(SalePrice) AS AverageSalePrice
FROM Sales;

```

Output:

```

35
36 • SELECT AVG(SalePrice) AS AverageSalePrice
37 FROM Sales;

```

Result Grid	
	AverageSalePrice
▶	254.000000

COUNT() - Count the number of sales made.

Unset

```

SELECT COUNT(*) AS TotalSales
FROM Sales;

```

Output:

```

38
39 • SELECT COUNT(*) AS TotalSales
40 FROM Sales;

```

	TotalSales
▶	5

MIN() - Find the lowest sale price.

Unset

```

SELECT MIN(SalePrice) AS MinimumSalePrice
FROM Sales;

```

Output:

```

41
42 • SELECT MIN(SalePrice) AS MinimumSalePrice
43 FROM Sales;

```

	MinimumSalePrice
▶	120.00

MAX() - Find the highest sale price.

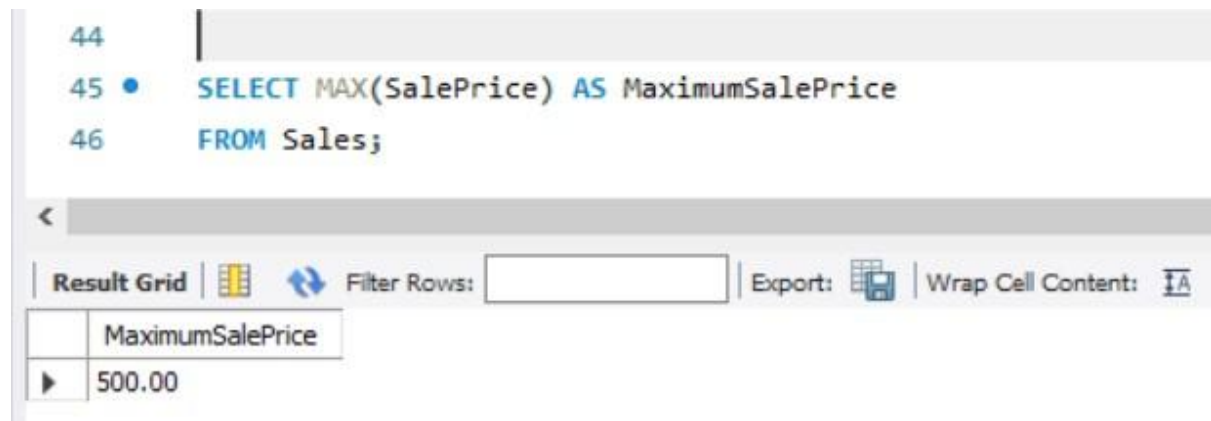
Unset

```

SELECT MAX(SalePrice) AS MaximumSalePrice
FROM Sales;

```

Output:



```

44
45 • SELECT MAX(SalePrice) AS MaximumSalePrice
46 FROM Sales;

```

MaximumSalePrice
500.00

String Functions

String functions are used to manipulate string data types. They allow for operations such as concatenation, extraction, and transformation of string values.

Common String Functions

- **CONCAT():**

This function joins two or more strings together into a single string. For example:

```

Unset
SELECT CONCAT(ProductName, ' - ', Category) AS ProductDetails
FROM Products;

```

This query retrieves a concatenated string of ProductName and Category from the Products table, formatted as "ProductName - Category". The resulting string is labelled as ProductDetails, providing a clear view of each product along with its category.

Output:

```
47 |
48 • SELECT CONCAT(ProductName, ' - ', Category) AS ProductDetails
49 FROM Products;
```

< |

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	ProductDetails
▶	Laptop - Electronics
	Smartphone - Electronics
	Tablet - Electronics
	Smartwatch - Electronics
	Headphones - Accessories

Result 6 x

- **SUBSTRING():**

Extracts a portion of a string based on specified starting position and length. For example:

Unset

```
SELECT ProductName, SUBSTRING(ProductName, 1, 5) AS ShortName
FROM Products;
```

This query selects the ProductName from the Products table and extracts the first five characters of each product name using the SUBSTRING function. The extracted portion is labelled as ShortName, allowing for a quick reference to a truncated version of each product name.

Output:

```
50 |
51 • SELECT ProductName, SUBSTRING(ProductName, 1, 5) AS ShortName
52 FROM Products;
```

< | Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ProductName	ShortName
▶	Laptop	Lapto
	Smartphone	Smart
	Tablet	Table
	Smartwatch	Smart
	Headphones	Headp

Result 7 x

- **LENGTH():**

Returns the length of a string in characters. For example:

Unset

```
SELECT ProductName, LENGTH(ProductName) AS NameLength
FROM Products;
```

This query retrieves the ProductName from the Products table and calculates the length of each product name using the LENGTH function. The length is labelled as NameLength, providing insight into how many characters each product name contains.

Output:

```

53
54 • SELECT ProductName, LENGTH(ProductName) AS NameLength
55 FROM Products;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ProductName	NameLength
▶	Laptop	6
	Smartphone	10
	Tablet	6
	Smartwatch	10
	Headphones	10

Result 8 x

• UPPER()/LOWER():

These functions convert strings to uppercase or lowercase, respectively. For example:

```

Unset
SELECT UPPER(ProductName) AS UpperCaseName, LOWER(ProductName) AS
LowerCaseName FROM Products;

```

This query selects the ProductName from the Products table and converts it to uppercase using the UPPER function and to lowercase using the LOWER function. The results are labelled as UpperCaseName and LowerCaseName, respectively, allowing for easy comparison between different text cases.

Output:

```

56 |
57 • SELECT UPPER(ProductName) AS UpperCaseName, LOWER(ProductName) AS LowerCaseName
58   FROM Products;

```

UpperCaseName	LowerCaseName
LAPTOP	laptop
SMARTPHONE	smartphone
TABLET	tablet
SMARTWATCH	smartwatch
HEADPHONES	headphones

Result 9 x

Date Functions

Date functions are designed to manipulate date values, enabling users to perform operations such as calculating differences between dates or formatting date values.

Key Date Functions

- **NOW():**

Returns the current date and time. For example:

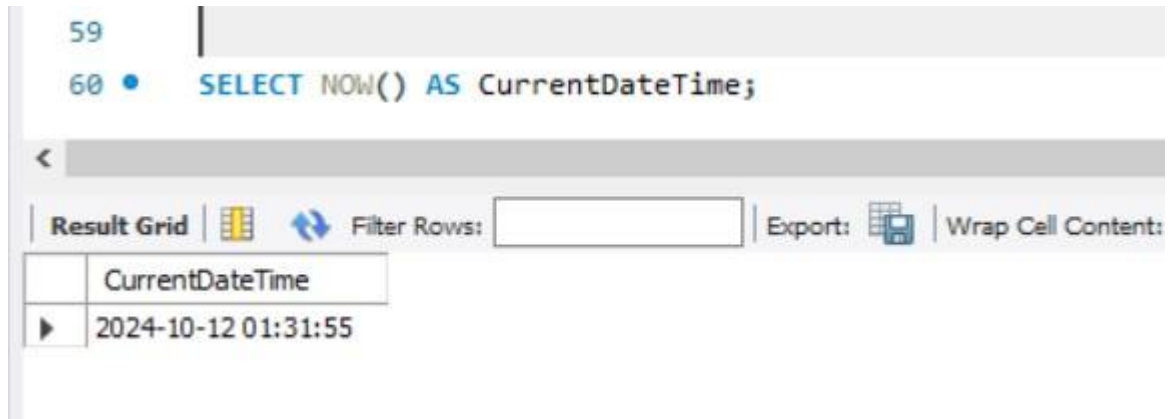
```

Unset
SELECT NOW() AS CurrentDateTime;

```

This query retrieves the current date and time from the system using the NOW() function. The result is labelled as CurrentDateTime, providing a timestamp of when the query was executed.

Output:



59 |

60 • `SELECT NOW() AS CurrentDateTime;`

< |

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content:

	CurrentDateTime
▶	2024-10-12 01:31:55

• CURDATE() / CURRENT_DATE()

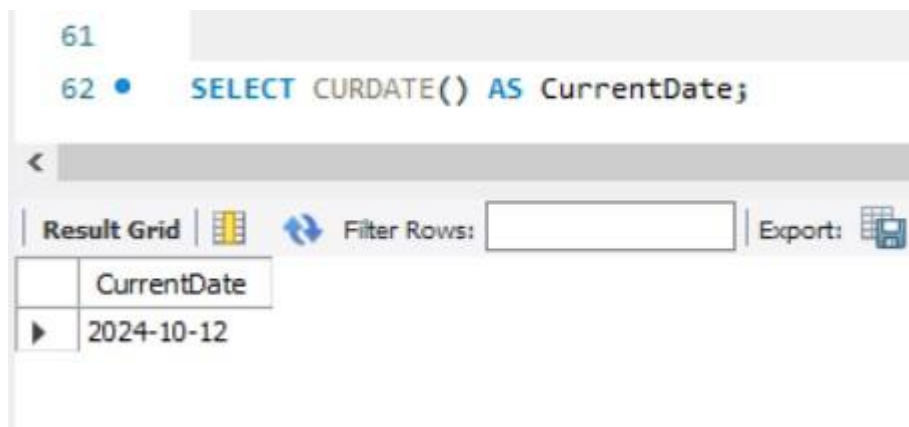
Returns the current date. Helpful for filtering records based on today's date or for logging purposes.

Unset

`SELECT CURDATE() AS CurrentDate;`

This query selects the current date (without the time) using the CURDATE() function. The result is labelled as CurrentDate, giving a straightforward representation of today's date.

Output:



61 |

62 • `SELECT CURDATE() AS CurrentDate;`

< |

Result Grid |   Filter Rows: | Export: 

	CurrentDate
▶	2024-10-12

- **DATEADD():**

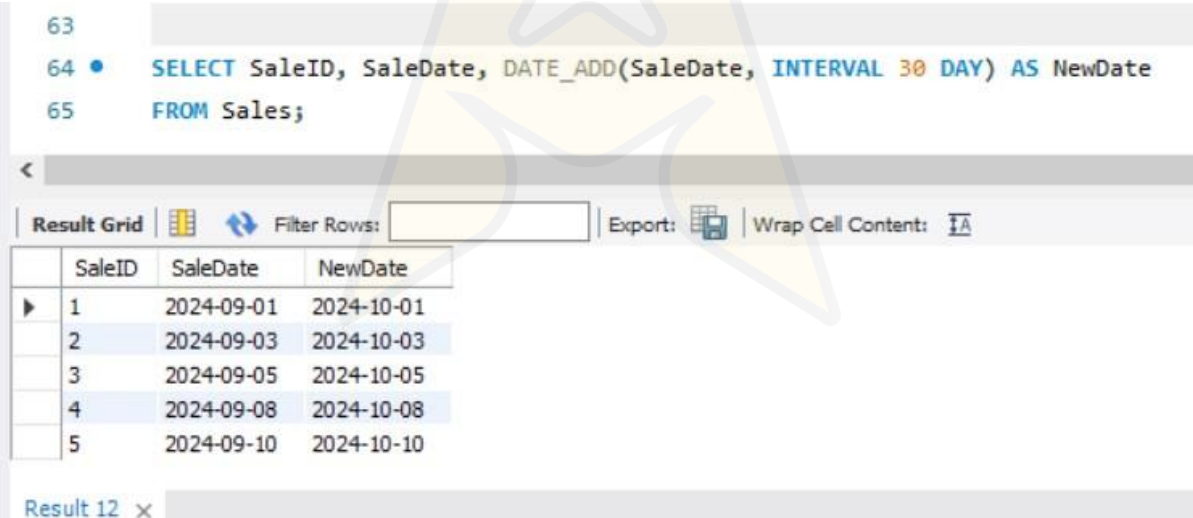
Adds a specified interval (like days or months) to a date. For example:

Unset

```
SELECT SaleID, SaleDate, DATE_ADD(SaleDate, INTERVAL 30 DAY) AS
NewDate
FROM Sales;
```

This query retrieves the SaleID and SaleDate from the Sales table and calculates a new date that is 30 days after each sale date using the DATE_ADD function. The resulting date is labelled as NewDate, providing insight into future dates based on sales.

Output:



The screenshot shows a SQL query editor with the following query:

```
63
64 • SELECT SaleID, SaleDate, DATE_ADD(SaleDate, INTERVAL 30 DAY) AS NewDate
65 FROM Sales;
```

Below the query, there is a 'Result Grid' showing the output of the query. The grid has columns for SaleID, SaleDate, and NewDate. The results are as follows:

	SaleID	SaleDate	NewDate
▶	1	2024-09-01	2024-10-01
	2	2024-09-03	2024-10-03
	3	2024-09-05	2024-10-05
	4	2024-09-08	2024-10-08
	5	2024-09-10	2024-10-10

At the bottom of the screenshot, it says 'Result 12 x'.

- **DATEDIFF():**

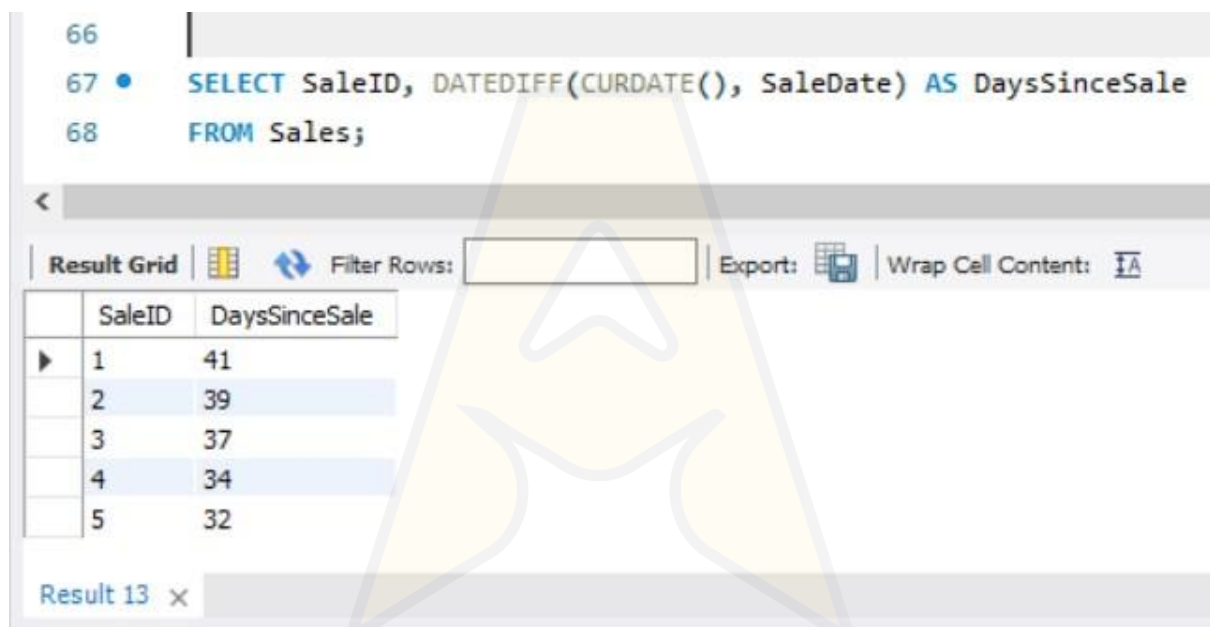
Calculates the difference between two dates and returns it as an integer. For example:

Unset

```
SELECT SaleID, DATEDIFF(CURDATE(), SaleDate) AS DaysSinceSale
FROM Sales;
```

This query selects the SaleID from the Sales table and calculates the number of days since each sale occurred by using the DATEDIFF function with the current date. The result is labelled as DaysSinceSale, indicating how long ago each sale took place.

Output:



The screenshot shows a SQL query editor with the following query:

```
66
67 • SELECT SaleID, DATEDIFF(CURDATE(), SaleDate) AS DaysSinceSale
68 FROM Sales;
```

Below the query, there is a toolbar with options: Result Grid, Filter Rows, Export, and Wrap Cell Content. The results are displayed in a table grid:

	SaleID	DaysSinceSale
▶	1	41
	2	39
	3	37
	4	34
	5	32

At the bottom, it says "Result 13 x".

• EXTRACT()

Extracts specific parts (like year, month, day) from a date. Useful for aggregating data by specific time periods.

Unset

```
SELECT SaleID, EXTRACT(YEAR FROM SaleDate) AS SaleYear
FROM Sales;
```

This query retrieves the SaleID from the Sales table and extracts the year from each SaleDate using the EXTRACT function. The extracted

year is labelled as SaleYear, providing a clear view of when each sale occurred in terms of its year.

Output:

```
69
70 • SELECT SaleID, EXTRACT(YEAR FROM SaleDate) AS SaleYear
71 FROM Sales;
```

	SaleID	SaleYear
▶	1	2024
	2	2024
	3	2024
	4	2024
	5	2024

Result 14 x

- **DATE_FORMAT()**

Formats a date according to a specified format. Useful for displaying dates in a user-friendly manner.

Unset

```
SELECT SaleID, DATE_FORMAT(SaleDate, '%m-%d-%Y') AS FormattedDate
FROM Sales;
```

This query selects the SaleID from the Sales table and formats each SaleDate using the DATE_FORMAT function to display it in the 'MM-DD-YYYY' format. The formatted date is labelled as FormattedDate, making it easier to read and interpret sale dates.

Output:

```

72
73 • SELECT SaleID, DATE_FORMAT(SaleDate, '%m-%d-%Y') AS FormattedDate
74 FROM Sales;

```

	SaleID	FormattedDate
▶	1	09-01-2024
	2	09-03-2024
	3	09-05-2024
	4	09-08-2024
	5	09-10-2024

Result 15 x

Mathematical Functions

Mathematical functions perform calculations on numeric data types, allowing for operations such as rounding or finding the smallest/largest integers.

- **ROUND():**

Rounds a number to a specified number of decimal places. For example:

Unset

```

SELECT SaleID, ROUND(SalePrice, 2) AS RoundedPrice
FROM Sales;

```

This query retrieves the SaleID and rounds the SalePrice to two decimal places using the ROUND function. The rounded price is labelled as RoundedPrice, providing a more precise representation of sale prices for reporting or display.

Output:

```

75
76 • SELECT SaleID, ROUND(SalePrice, 2) AS RoundedPrice
77 FROM Sales;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	SaleID	RoundedPrice
▶	1	150.00
	2	300.00
	3	500.00
	4	200.00
	5	120.00

Result 16 x

• FLOOR():

Returns the largest integer less than or equal to a given number.

For example:

```

Unset
SELECT SaleID, FLOOR(SalePrice) AS FlooredPrice
FROM Sales;

```

This query selects the SaleID and applies the FLOOR function to the SalePrice, which returns the largest integer less than or equal to the sale price. The result is labelled as FlooredPrice, effectively truncating any decimal portion of the sale price.

Output:

```

78
79 • SELECT SaleID, FLOOR(SalePrice) AS FlooredPrice
80 FROM Sales;

```

<

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	SaleID	FlooredPrice
▶	1	150
	2	300
	3	500
	4	200
	5	120

Result 17 x

- **CEIL():**

Returns the smallest integer greater than or equal to a given number. For example:

Unset

```

SELECT SaleID, CEIL(SalePrice) AS CeilingPrice
FROM Sales;

```

This query retrieves the SaleID and uses the CEIL function on the SalePrice, returning the smallest integer greater than or equal to the sale price. The result is labelled as CeilingPrice, rounding up any decimal portion of the sale price.

Output:

```

81
82 • SELECT SaleID, CEIL(SalePrice) AS CeilingPrice
83 FROM Sales;

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	SaleID	CeilingPrice
▶	1	150
	2	300
	3	500
	4	200
	5	120

Result 18 x

- **ABS():**

Returns the absolute value of a number. For example:

Unset

```

SELECT ABS(300 - SalePrice) AS PriceDifference
FROM Sales;

```

This query calculates the absolute difference between 300 and each SalePrice using the ABS function. The result is labelled as PriceDifference, providing a measure of how far each sale price deviates from 300, regardless of whether it is above or below that value.

Output:

```

84
85 • SELECT ABS(300 - SalePrice) AS PriceDifference
86 FROM Sales;

```

<
 Result Grid
 Filter Rows:
 Export:
 Wrap Cell Content:

	PriceDifference
▶	150.00
	0.00
	200.00
	100.00
	180.00

Result 19 x

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)